Section 1: Project Planning & Management

Project: STAR Cinema — Online Movie Seat Booking Web Application

---

1. Project Proposal

Overview

STAR Cinema is a full-stack web application for booking cinema seats. Users can browse movies, view upcoming showtimes, select a seat from an 80-seat screen layout (rows A–H, seats 1–10), and book, view or cancel their bookings after registering and logging in.

The system has three parts:

- a REST API built with Node.js and Express;
- a MySQL database;
- a browser frontend built with HTML, CSS and JavaScript.

Users are authenticated with JSON Web Tokens (JWT), and the application can be run locally or with Docker Compose.

Objectives

1. Allow authenticated users to book a seat for a showtime, and prevent the same seat from being booked twice.
2. Allow users to view and cancel their own bookings.
3. Store passwords securely (bcrypt hashing) and protect booking operations with JWT authentication.
4. Validate all user input on the server and use parameterised SQL queries.
5. Provide reproducible setup (Docker Compose and a local setup script) and automated tests.

Scope

In scope	Out of scope
User registration and login	Online payment and e-tickets
Movie catalogue and search	Administration panel (data is loaded through SQL seed scripts)
Showtimes and seat availability	Multiple cinemas and seat categories
Seat booking, booking history and cancellation	Email verification and password reset
Docker Compose deployment and health check	Mobile applications

Current implementation status:

- **Backend API:** implements every in-scope feature.

- **Frontend:** registration and login are connected to the API. The movie, showtime and booking pages still use demo data and do not yet save bookings to the database.

---

2. Project Plan

Timeline and Gantt Chart

The timeline below is aligned with the official DEPI deadlines. Start dates are planned.

```
gantt
    title STAR Cinema — Project Timeline
    dateFormat YYYY-MM-DD
    axisFormat %d %b
    section Documentation
    Project Planning & Management          :p1, 2026-10-01, 2026-10-16
    Literature Review                      :p2, 2026-10-03, 2026-10-16
    Requirements Gathering                 :p3, 2026-10-05, 2026-10-16
    System Analysis & Design               :p4, 2026-10-17, 2026-11-06
    section Development
    Implementation (Source Code & Execution) :p5, 2026-10-01, 2026-11-30
    section Final Phase
    Testing                                :p6, 2026-11-07, 2026-12-04
    Final Presentation & Reports           :p7, 2026-11-23, 2026-12-04
```

Milestones and Deliverables

Milestone / Deliverable	Industry deadline	DEPI deadline	Status
Project Planning & Management	2 October 2026	16 October 2026	Completed (this document)
Literature Review	9 October 2026	16 October 2026	Completed
Requirements Gathering	9 October 2026	16 October 2026	Planned
System Analysis & Design	9 October 2026	6 November 2026	Planned
Implementation (Source Code & Execution)	16 October 2026	30 November 2026	In progress: backend implemented; frontend integration pending
Final Presentation, Testing & Reports	16 October 2026	4 December 2026	Planned

Resource Allocation

Phase	Lead	Support	Tools
Planning & Literature Review	أسامه زكى محمد فرحات	All members	Markdown, GitHub
Requirements, Analysis & Design	امل فتحي عبدالحميد علي	أسامه زكى محمد فرحات, بيشوي ماجد حنا ابراهيم	Markdown, diagram tools
Backend & Database	احمد محمود عبدالقادر مراد	امل فتحي عبدالحميد علي	Node.js, Express, MySQL
Frontend	براء صبري احمد حسين	احمد محمود عبدالقادر مراد	HTML, CSS, JavaScript

Phase	Lead	Support	Tools
Security & Deployment	بيشوي ماجد حنا ابراهيم	احمد محمود عبدالقادر مراد	Docker Compose, Helmet
Testing & Final Presentation	مي محمد صابر السيد	All members	Node.js test runner, browser testing

---

3. Task Assignment & Roles

Team member	Role	Responsibilities	Deliverables
أسامه زكى محمد فرحات	Project Coordinator	Planning, scheduling, tracking deadlines, compiling documentation	Sections 1–2, project plan, final presentation
احمد محمود عبدالقادر مراد	Backend Developer	API endpoints, booking logic, server configuration	Completed backend API
امل فتحي عبدالحميد علي	Database & Requirements Analyst	Database schema, seed data, requirements gathering	Database scripts, Section 3 (Requirements Gathering)
براء صبري احمد حسين	Frontend Developer	Connecting pages to the API, seat selection, booking history and cancellation interface	Completed frontend
بيشوي ماجد حنا ابراهيم	Security & DevOps Engineer	Risk management, authentication security, Docker setup	Risk plan, secured configuration, verified Docker deployment
مي محمد صابر السيد	QA & Testing Engineer	KPI tracking, integration and browser testing, test reporting	Test suite, test report

All members contribute to Section 4 (System Analysis & Design) and the final presentation.

---

4. Risk Assessment & Mitigation Plan

Risk	Impact	Mitigation
Frontend pages not yet connected to the backend API	High	Prioritise connecting the movie, showtime and booking pages to the existing API endpoints
Booking page script blocked by the server's Content Security Policy (inline script)	High	Move inline scripts and styles in `booking.html` into external files
Existing tests check file contents only, not application behaviour	High	Add API integration tests with a test database and browser tests for the booking flow
Tight schedule for the remaining deliverables	Medium	Weekly progress reviews against the timeline; prioritise core features

Risk	Impact	Mitigation
Demo showtimes expire 1–3 days after the database is seeded	Medium	Update the seed script to refresh showtime dates; re-seed before the demonstration
Security weaknesses (default fallback secrets, no login rate limiting)	Medium	Remove hard-coded secrets, add rate limiting and strengthen the password policy
Different Node.js and MySQL versions across environments	Medium	Standardise versions and use Docker Compose for the demonstration
Posters and fonts loaded from external services	Low	Test the demonstration network in advance and provide local fallback images

---

5. KPIs (Key Performance Indicators)

KPI	Measurement method	Target
API response time	Measure response time of the main endpoints during testing	< 500 ms for 95 % of requests
Test pass rate	Automated integration and browser test results	100 % before final submission
Double-booking prevention	Simultaneous booking requests for the same seat	Exactly one booking succeeds
Booking flow success rate	Browser test of register → log in → book → view → cancel	All steps succeed
System uptime	Monitor the `/api/health` endpoint during the demonstration period	≥ 99 %
Error rate	Share of server errors (HTTP 5xx) during testing	< 1 %
Milestone completion	Deliverables submitted by the official DEPI deadlines	100 %

Actual results will be measured and reported during the Testing phase.

Section 2: Literature Review

Project: STAR Cinema — Online Movie Seat Booking Web Application

1. Feedback & Evaluation

At the time of writing, no formal lecturer feedback or evaluation has been received. This section will be updated once the lecturer reviews the project and provides comments or recommendations.

2. Suggested Improvements

#	Improvement	Expected benefit
1	Connect the frontend pages to the existing API endpoints (movies, showtimes, seats, bookings, history, cancellation)	A complete booking journey from the browser
2	Move the inline script and styles in booking.html into external files	The booking page works under the existing Content Security Policy, without weakening it
3	Add API integration tests with a test database, and browser tests for the booking flow	Tests verify real behaviour and catch broken features
4	Update the seed script so that showtime dates refresh each time it runs	Bookable showtimes are always available for demonstration
5	Remove hard-coded fallback secrets and demo passwords from code and scripts	Lower risk of token forgery and credential exposure [1]
6	Add rate limiting to login and registration	Protection against brute-force and credential-stuffing attacks [2]
7	Increase the minimum password length, and enforce bcrypt's 72-byte limit in bytes rather than characters	Alignment with current password guidance [3], [4]
8	Update the README to reflect actual behaviour, and remove leftover backup files	Accurate, maintainable documentation

References

[1] Y. Sheffer, D. Hardt, and M. Jones, "JSON Web Token Best Current Practices," IETF RFC 8725, Feb. 2020. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8725

[2] OWASP Foundation, "OWASP API Security Top 10 — 2023." [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0x00-header/

[3] OWASP Foundation, "Password Storage Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[4] NIST, "Digital Identity Guidelines: Authentication and Authenticator Management," SP 800-63B-4, Jul. 2025, doi: 10.6028/NIST.SP.800-63b-4.

3. Final Grading Criteria

No official grading breakdown was included in the supplied materials. The final grade will be based on the four assessment areas specified in the DEPI guidelines:

1. Documentation
2. Implementation
3. Testing
4. Presentation

3. Requirement Gathering

3.1 Stakeholder Analysis

Eight stakeholder groups:

Stakeholder	Role	Their needs
Customer	Primary end user who books tickets	Easy browsing, fast booking, clear price, seat choice, booking confirmation, cancellation
Cinema management / owner	Sponsor who funds and owns the business outcome	Higher ticket sales, accurate reports, low running cost, no double bookings
Cinema admin	Maintains films, showtimes, halls and prices	Simple screens to add and edit films and showtimes, view bookings
Box-office / entrance staff	Sells tickets on site and checks them at the door	Quick ticket lookup and validation, view current seat availability
Development team	Builds the frontend, backend and database	Clear requirements, version control, automated tests, fast feedback
DevOps team	Deploys, scales, monitors and recovers the system	Repeatable deployments, dashboards and alerts, safe rollback, infrastructure as code
Payment provider (external)	Processes online payments, if used	Correct and secure API usage, no stored card data
Project supervisor	Reviews the project and its deliverables	Complete documentation, working pipeline, demonstrable system

3.2 User Stories

The stories below cover the four user groups and the DevOps team, each with acceptance criteria the team can test.

Customer

ID	User story
US-01	As a customer, I want to create an account and log in, so that my bookings are saved under my name.
US-02	As a customer, I want to browse films now showing, so that I can choose what to watch.
US-03	As a customer, I want to see the showtimes for a film, so that I can pick a convenient time.
US-04	As a customer, I want to select my seats on a seat map, so that I sit where I prefer.
US-05	As a customer, I want to confirm my booking and receive a confirmation, so that I know my seats are reserved.
US-06	As a customer, I want to cancel a booking, so that I can change my plans.

Cinema admin

ID	User story
US-07	As an admin, I want to add, edit and remove films, so that the catalogue is current.
US-08	As an admin, I want to schedule showtimes for each hall, so that customers can book them.
US-09	As an admin, I want to view bookings and sales reports, so that I can track revenue.

Box-office staff

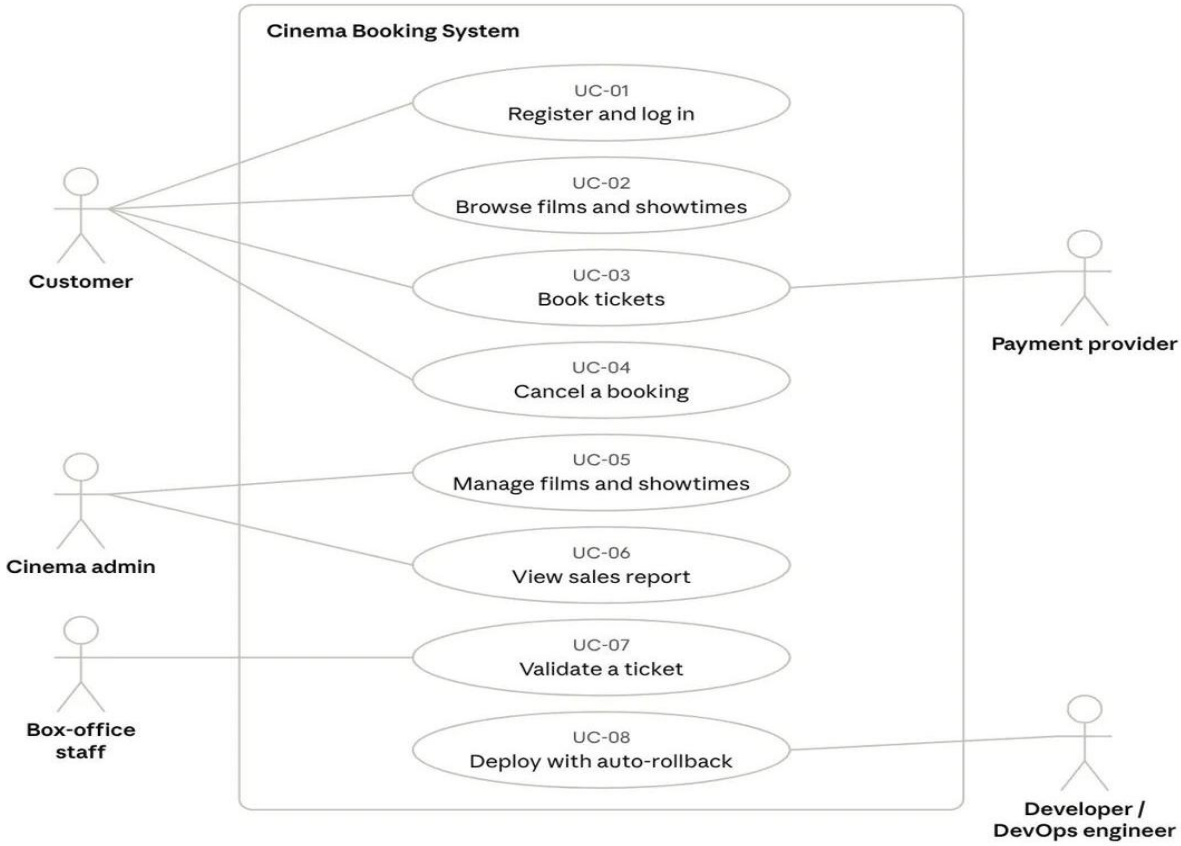
ID	User story
US-10	As a staff member, I want to look up a booking by its reference, so that I can validate a ticket at the door.

DevOps team

ID	User story
US-11	As a developer, I want every push to Git to trigger a Jenkins build and tests, so that errors are found early.
US-12	As a DevOps engineer, I want the application to be deployed as Docker containers on Kubernetes, so that releases are repeatable.
US-13	As a DevOps engineer, I want the system to scale automatically under load, so that booking stays fast on busy days.
US-14	As a DevOps engineer, I want dashboards and alerts for service health, so that I can react before customers notice.
US-15	As a DevOps engineer, I want a failed release to roll back automatically, so that downtime is short.
US-16	As a DevOps engineer, I want the AWS infrastructure and servers defined as code, so that environments can be rebuilt consistently.

3.2 Use Cases

UMl Digaram and eight use cases describe how users and the DevOps team interact with the system. Each one lists the use case, actor, trigger, pre-condition, main success scenario, post condition and extensions.



Use case	Register and log in
Actor	Customer
Trigger	The customer opens the site and selects Sign up or Log in.
Pre-condition	The customer is not logged in. To log in, they already have an account, and the frontend, backend and database are running
Main success scenario:	1. The customer enters name, email and password. 2. The frontend sends the data to the backend. 3. The backend validates the data and checks that the email is not used. 4. The backend stores the user with a hashed password in MySQL. 5. The customer logs in with email and password. 6. The backend verifies the credentials and returns a session token. 7. The system shows the home page as a logged-in user.
Post condition	The customer has an account and an active session.
Extensions:	o 3a. Email already registered: the system shows an error and offers login. o 3b. Invalid data (weak password, bad email format): the system shows which field to fix. o 6a. Wrong credentials: the system shows an error and, after repeated failures, locks the login for a short time.

Use case	Browse films and showtimes
Actor	Customer (guest or logged in)
Trigger	The customer opens the home page or the films page.
Pre-condition	The system is running and at least one film is in the catalogue. No login is needed.
Main success scenario:	1. The frontend requests the list of films from the backend. 2. The backend reads the films from MySQL and returns them. 3. The customer selects a film. 4. The system shows the film details and its upcoming showtimes with hall, time, price and free seats.
Post condition	The customer can see the film and showtime details. No data is changed
Extensions:	o 2a. Database or backend unavailable: the system shows a friendly error and asks the customer to try again. o 4a. The film has no upcoming showtimes: the system shows that none are scheduled.

Use case	Book tickets
Actor	Customer (logged in)
Trigger	The customer selects a showtime and presses Book.
Pre-condition	The customer selects a showtime and presses Book.
Main success scenario:	1. The frontend shows the seat map for the showtime. 2. The customer selects one or more seats and presses Book. 3. The frontend sends the booking request (POST) to the backend API. 4. The business logic checks that the showtime exists, the seats are free and the user is allowed to book. 5. The business logic calculates the total price. 6. The backend saves the booking and marks the seats as taken in MySQL, in one transaction. 7. The system returns the result and shows a confirmation with the booking reference.
Post condition	The booking is stored, the seats are reserved and no other customer can book them.
Extensions:	o 3a. Customer is not logged in: the system redirects to login and returns to the seat map afterwards. o 4a. A selected seat was taken by someone else meanwhile: the system rejects the request, refreshes the seat map and asks the customer to choose again. o 4b. The showtime has passed or does not exist: the system shows an error and returns to the film page. o 6a. Database error: the transaction is rolled back, no seats are held and the customer is asked to retry. o 7a. Payment is required and fails: the booking is cancelled and the seats are released.

Use case	Cancel a booking
Actor	Customer (logged in)
Trigger	The customer selects Cancel on a booking in their booking history.
Pre-condition	The customer is logged in and has a confirmed booking for a future showtime that is not already cancelled or used.
Main success scenario:	1. The system shows the booking details and asks for confirmation. 2. The customer confirms. 3. The backend checks that the booking belongs to the customer and is before the cancellation cutoff. 4. The backend marks the booking as cancelled and releases the seats. 5. The system shows a cancellation confirmation.
Post condition	The booking status is cancelled and the seats are available again.
Extensions:	o 2a. The customer declines: nothing changes. o 3a. The cutoff time has passed: the system refuses and explains why. o 3b. The booking does not belong to the customer: the system denies access.

Use case	Manage films and showtimes
Actor	Cinema admin
Trigger	The admin opens the admin panel and selects Films or Showtimes.
Pre-condition	The admin has an account with the admin role. For a showtime, the film and hall already exist.
Main success scenario:	1. The admin logs in and the system confirms the admin role. 2. The admin chooses to add, edit or remove a film or showtime. 3. The admin enters the details (title, duration, hall, date, time, price). 4. The backend validates the data. 5. The backend saves the change in MySQL. 6. The system shows the updated list.
Post condition	The catalogue and schedule are updated and visible to customers
Extensions:	o 1a. The user is not an admin: the system denies access. o 4a. The showtime overlaps another one in the same hall: the system rejects it and shows the conflict. o 5a. The admin removes a showtime that already has bookings: the system blocks the removal or asks to cancel and notify those bookings first.

Use case	View sales report
Actor	Cinema admin (management may also use it)
Trigger	The admin selects Reports and chooses a date range.
Pre-condition	The user is logged in as admin or management, and bookings exist for the chosen period

Main success scenario:	1. The admin selects the date range and optional film filter. 2. The backend queries bookings in MySQL. 3. The backend totals tickets sold and revenue per film and per day. 4. The system displays the report.
Post condition	The report is shown. No data is changed.
Extensions:	o 2a. No bookings in the range: the system shows an empty report. o 1a. Invalid date range: the system asks for a correct range.

Use case	Validate a ticket
Actor	Box-office / entrance staff
Trigger	A customer presents a booking reference at the cinema.
Pre-condition	The staff member is logged in with the staff role and the customer has a booking reference.
Main success scenario:	1. The staff member enters the booking reference. 2. The backend finds the booking and checks it is confirmed and for the current showtime. 3. The system shows the booking details as valid. 4. The staff member confirms entry and the backend marks the booking as used.
Post condition	The booking is marked as used and cannot be reused.
Extensions:	o 2a. Reference not found: the system shows invalid ticket. o 2b. Booking cancelled or already used: the system shows the status and refuses entry. o 2c. Booking is for a different showtime: the system shows the correct showtime.

Use case	Deploy a release with automatic rollback
Actor	Developer, DevOps engineer (Jenkins acts as the automated system)
Trigger	A developer pushes or merges code to the main branch in Git.
Pre-condition	The code is in Git and Jenkins has access to Git, the image registry and the Kubernetes cluster. The AWS infrastructure is already provisioned with Terraform and Ansible, and a previous version is running.
Main success scenario:	Jenkins detects the change and checks out the code. Jenkins runs the build and automated tests. Jenkins builds the Docker image and pushes it to the image registry with a version tag. Jenkins updates the Kubernetes deployment with the new image (rolling update). Kubernetes starts new pods and routes traffic through Nginx and the load balancer once they pass health checks. Prometheus and Grafana show healthy metrics for the new version. Jenkins marks the release as successful and notifies the team.
Post condition	The new version is running in production and the previous version is kept for rollback.
Extensions:	2a. Build or tests fail: the pipeline stops, nothing is deployed and the team is notified. 5a. New pods fail health checks: Kubernetes keeps the old pods serving traffic and the pipeline triggers a rollback to the previous image. 6a. Error rate or latency passes the alert threshold after release: an alert is sent and the automated rollback restores the previous version. 4a. Cluster has no free capacity: the auto-scaler adds nodes or the deployment waits and the team is alerted.

3.3 Functional Requirements

Customer	* View available movies. * View movie details. * Select showtime.	* View seat availability. * Book available seats. * Receive booking confirmation.
Cinema Manager	* Add new movies. * Update movie information.	* Manage showtimes. * View booking statistics.
Administrator	* Deploy application using Kubernetes. * Configure servers using Ansible.	* Monitor application using Prometheus and Grafana. * Automate deployment using Jenkins.

3.4 Non-Functional Requirements

The system must stay fast during peak booking, protect user data, be easy to use and keep running through failures. The targets below are proposed starting values for the team to confirm and measure with Prometheus and Grafana.

Performance

Requirement	Target
Page and API response time for browsing films and showtimes	95% of requests under 2 seconds

Requirement	Target
Booking request processing time	95% of bookings completed under 3 seconds
Concurrent users supported	Rising automatically through Kubernetes auto-scaling
Scaling reaction	New pods start within 2 minutes of CPU use passing the threshold (for example 70%)
Database queries	Indexes on showtime, seat and booking lookups; no query over 500 ms under normal load

Security

Requirement	Target
Passwords	Stored only as salted hashes, never in plain text
Data in transit	All traffic over HTTPS/TLS, terminated at Nginx or the load balancer
Access control	Role-based: customers see only their own bookings; admin and staff functions need the matching role
Secrets	Passwords, keys and tokens kept out of Git; held in Jenkins credentials, Kubernetes secrets or AWS services
Infrastructure	Database in a private subnet; security groups allow only needed ports; least-privilege IAM roles
Payment data	Card details are never stored by the system

Usability

Requirement	Target
Booking effort	A new user completes a booking in 5 steps or fewer after choosing a film
Devices	Responsive layout that works on phones, tablets and desktops
Feedback	Clear error and success messages for every action; seat availability shown visually
Browsers	Latest versions of Chrome, Edge, Firefox and Safari
Language and accessibility	Readable contrast and keyboard-usable forms; Arabic and English support as an optional extension

Reliability

Requirement	Target
Availability	99.5% uptime per month, excluding planned maintenance
No double booking	A seat can be booked once only, enforced by database transactions and constraints
Fault tolerance	At least 2 application replicas behind the load balancer; failed pods restart automatically
Backup and recovery	Daily RDS backups; data restorable within 4 hours; no more than 24 hours of data lost
Rollback	A failed release is rolled back automatically within 5 minutes
Monitoring and alerting	Prometheus collects metrics and Grafana dashboards show request rate, errors, latency and resource use; alerts fire on failures

DevOps and maintainability

Requirement	Target
Reproducible environments	Infrastructure defined in Terraform and server setup in Ansible, both versioned in Git
Automated delivery	Every merge triggers the Jenkins build, test and deploy pipeline with no manual deployment steps
Portability	Application runs in Docker containers with the same image in every environment
Code quality	Code review before merge to the main branch; a minimum share of code covered by automated tests, agreed by the team